PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

ГАЗИНФОРМ СЕРВИС

Масштабирование в Citus

Александр Календарёв
разработчик СУБД Jatoba

www.gaz-is.ru www.jatoba.ru

Москва, 19 марта 2026

Инсталляция

shared_preload_libraries = `citus`

CREATE EXTENSION citus

```
test=# \dx
                    List of installed extensions
      Name       | Version |   Schema   |         Description
----------------+---------+------------+-----------------------------
 citus           | 12.1-1  | pg_catalog | Citus distributed database
 citus_columnar  | 11.3-1  | pg_catalog | Citus Columnar extension
 plpgsql         | 1.0     | pg_catalog | PL/pgSQL procedural language
(3 rows)
```

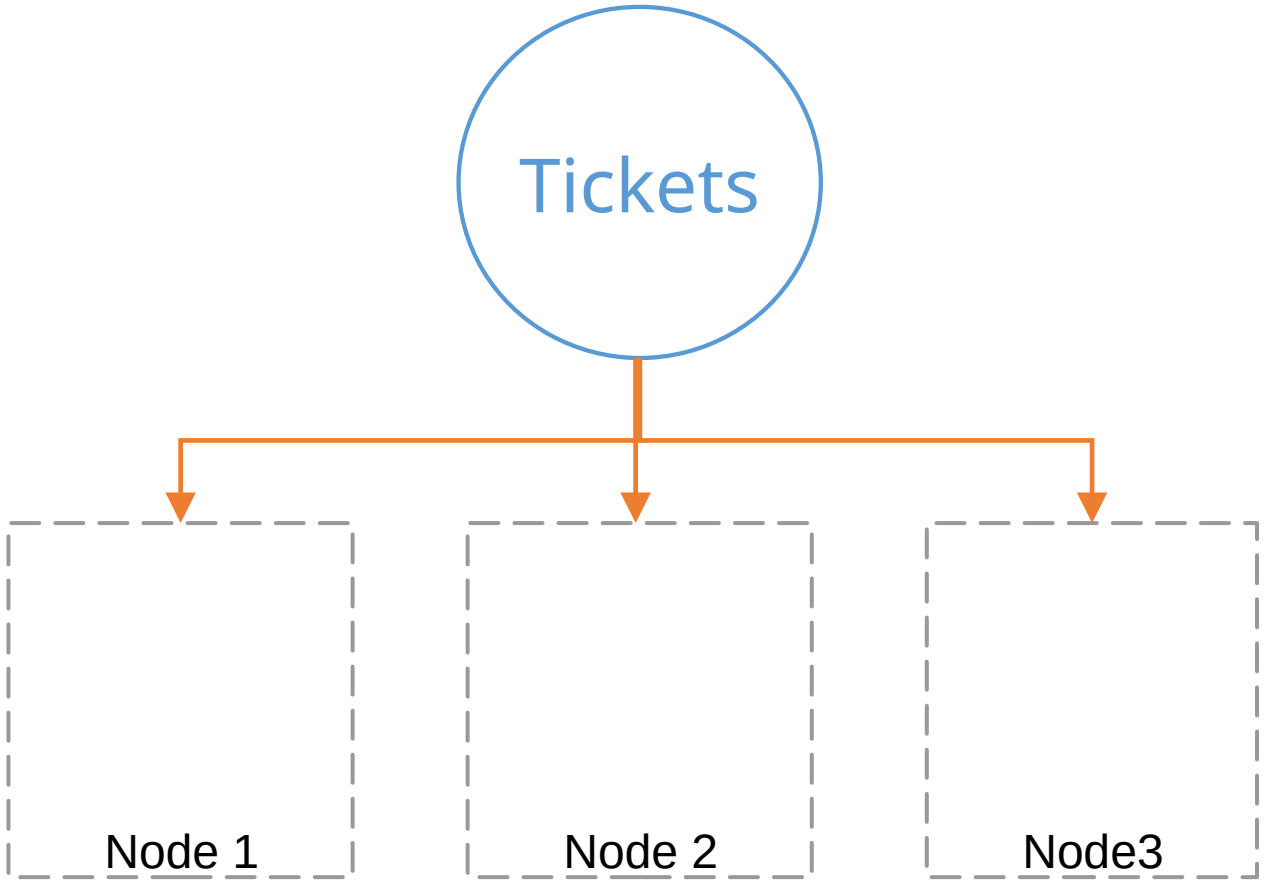
Citus columnar

- Эффективен для разряженных данных
- Быстрее расчет агрегатных функций (count, avg, sum)
- Встроенное сжатие
- Эффективнее работает VACUUM
- Эффективнее осуществляются I/O операции
- Нет изменения данных (append-only)
- Нет поддержки строковых блокировок

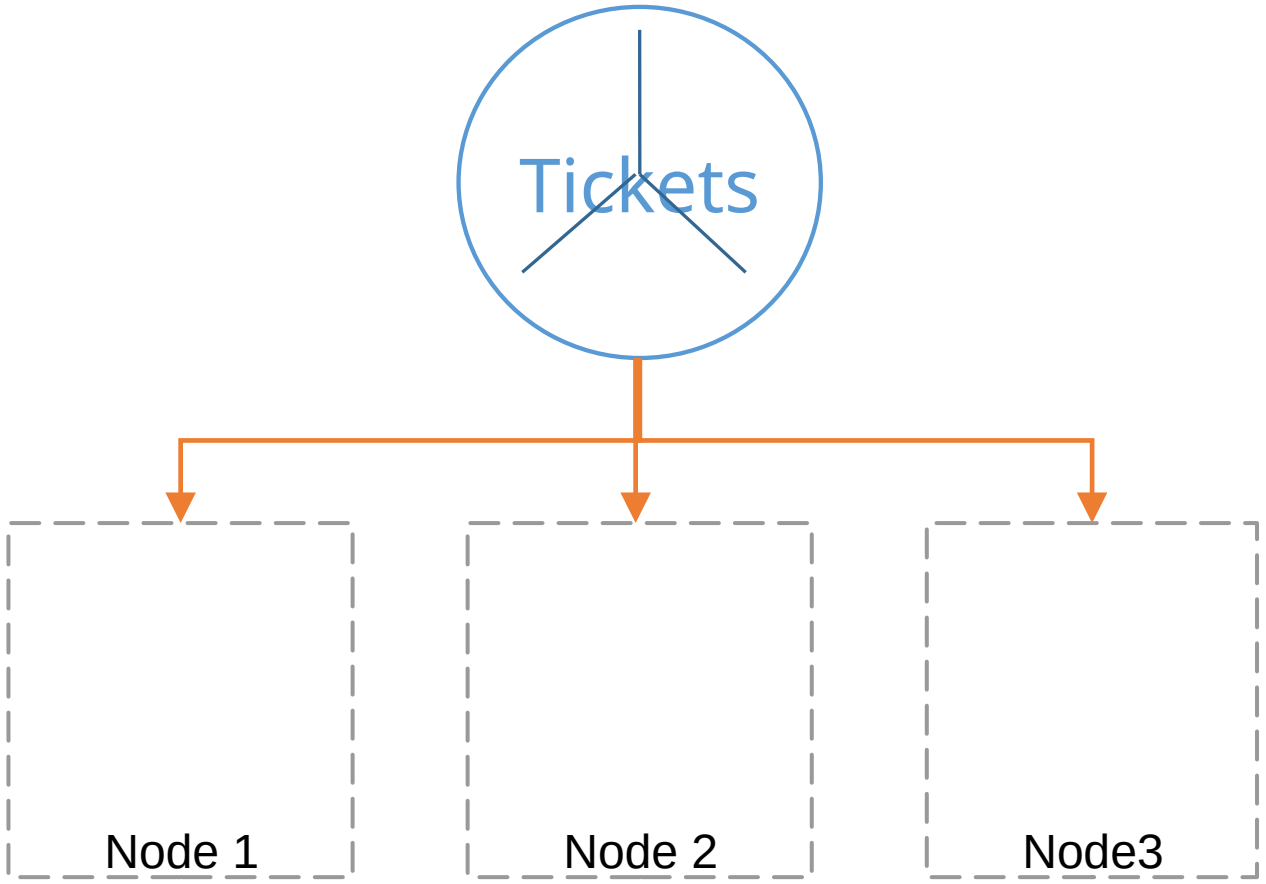
Какие задачи мы хотим решить?

- Распределение данных
- Возможность осуществлять распределенные запросы
- Согласованность данных
- Отказоустойчивость

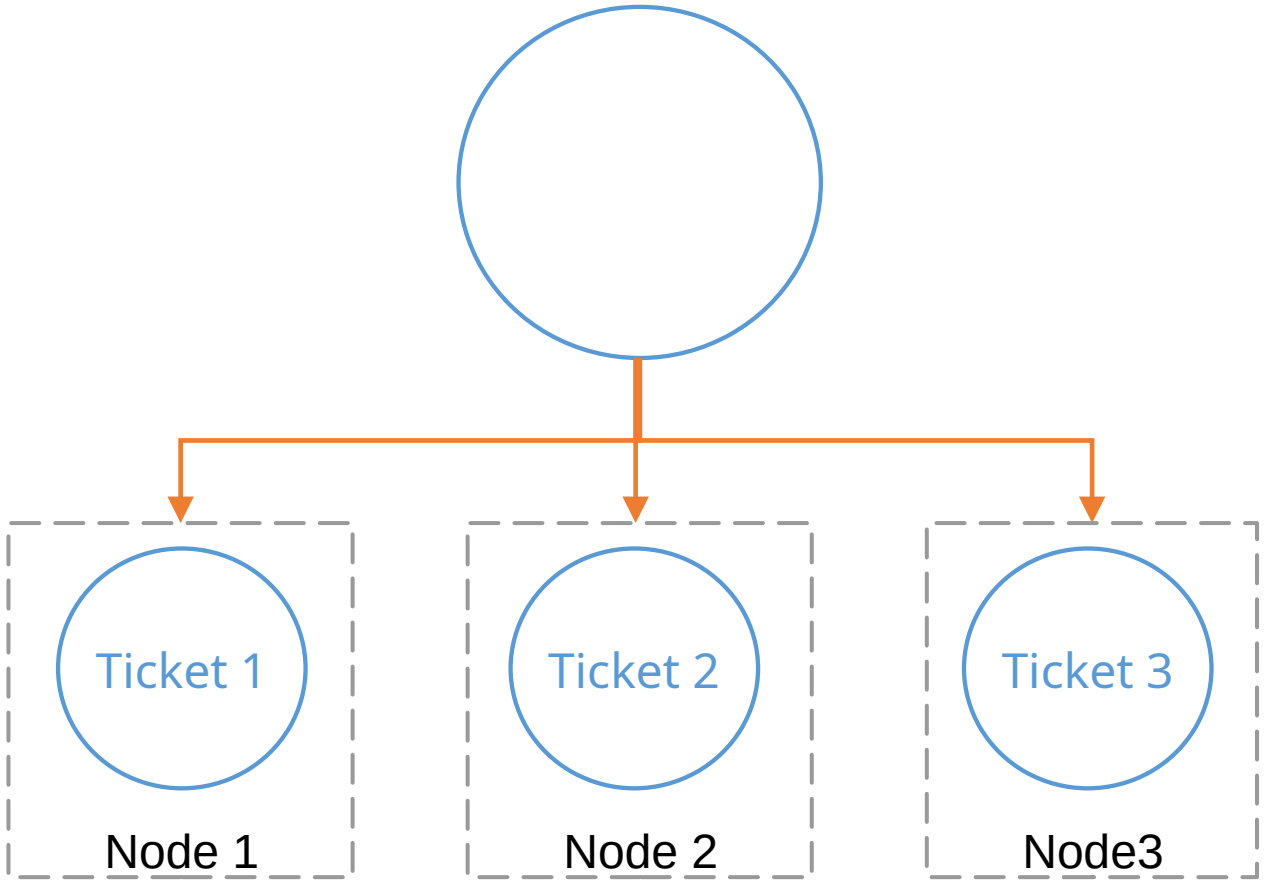
Процесс масштабирования данных



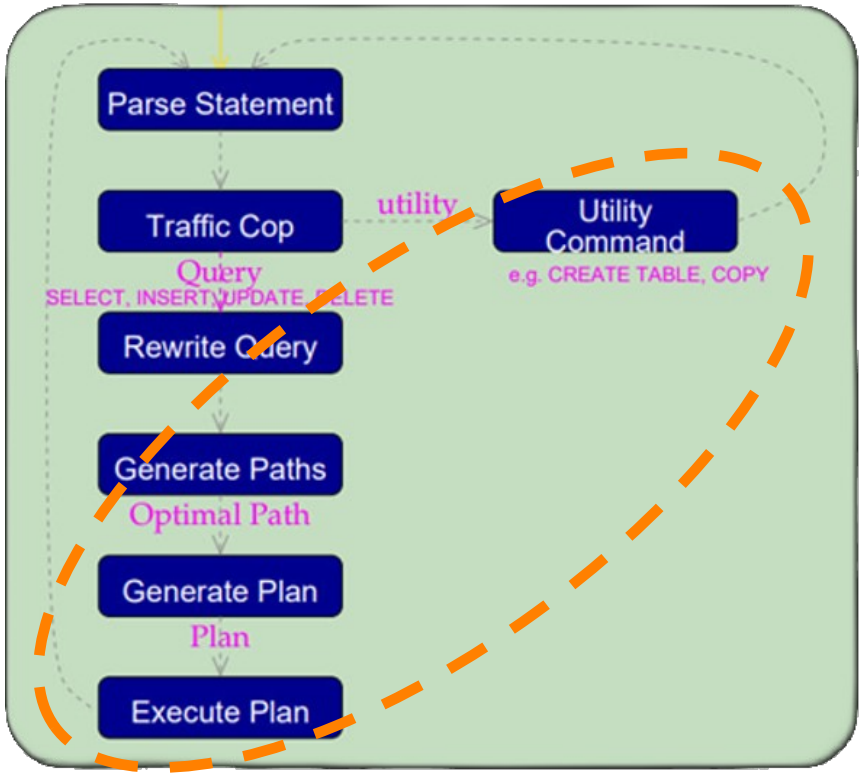
Процесс масштабирования данных



Процесс масштабирования данных

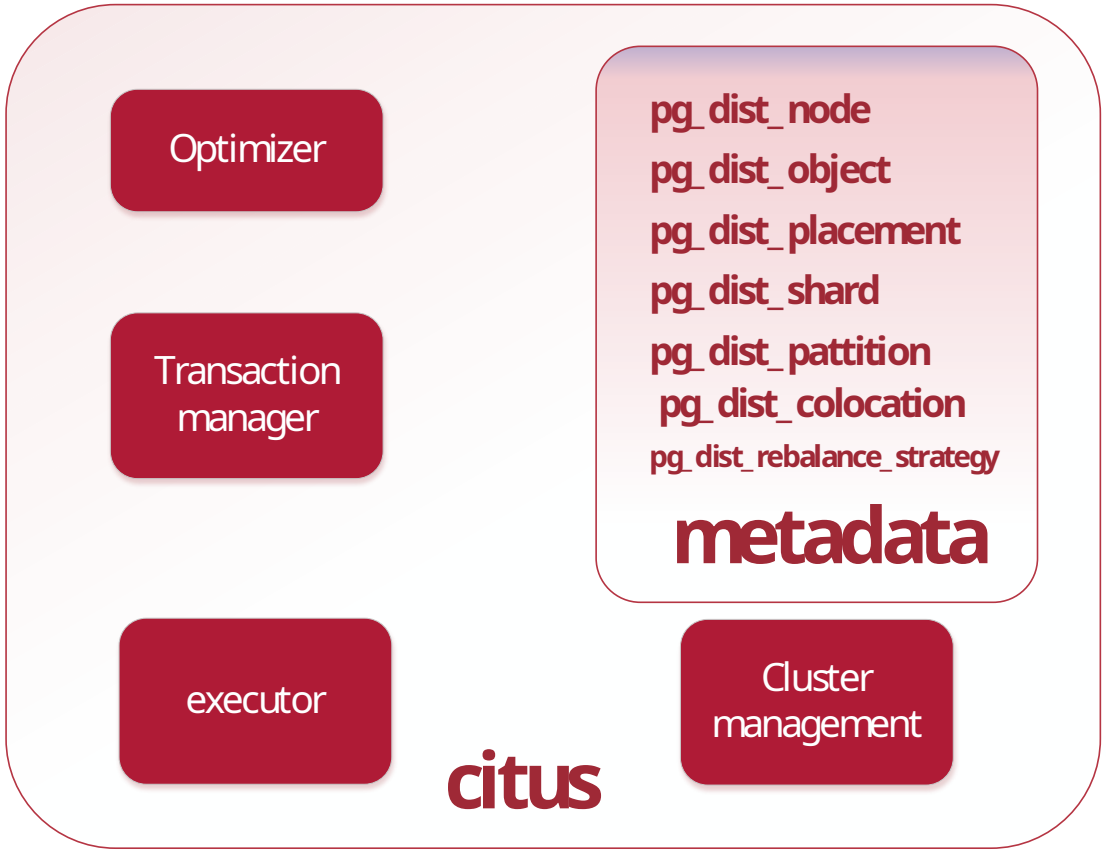


Как устроен postgres

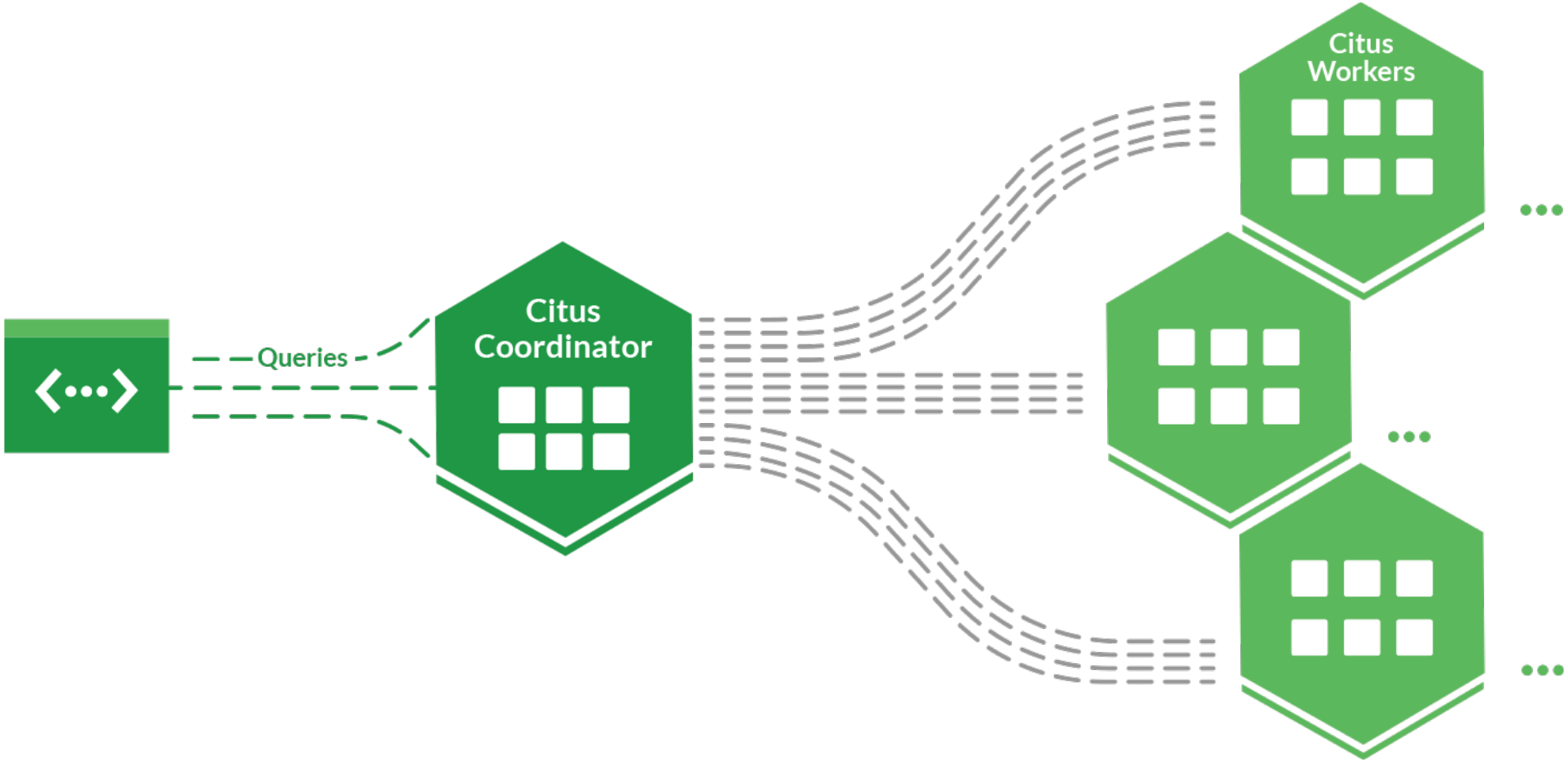


https://www.postgresql.org/developer/backend

Как устроен Citus



Взгляд команды Citus на кластер

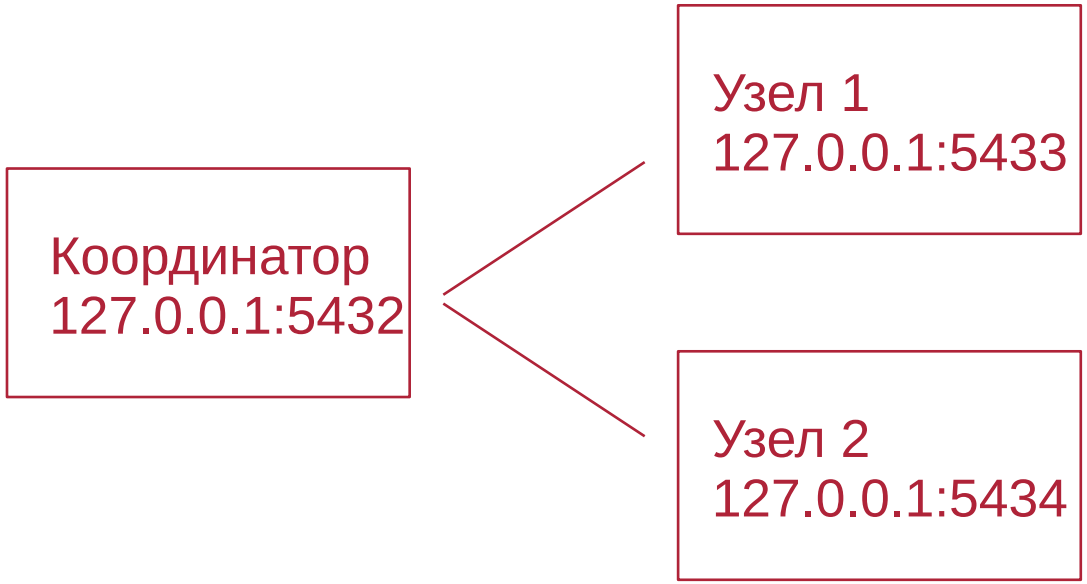


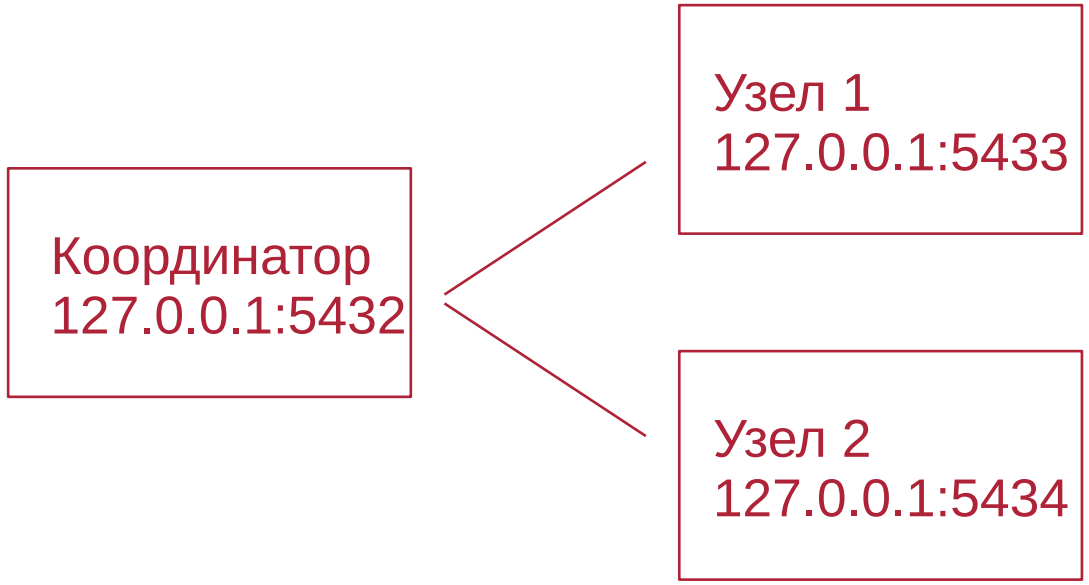
https://github.com/citusdata/citus/blob/main/images/citus-architecture.png

ГАЗИНФОРМ СЕРВИС

координатор и метаданные

Создаём кластер

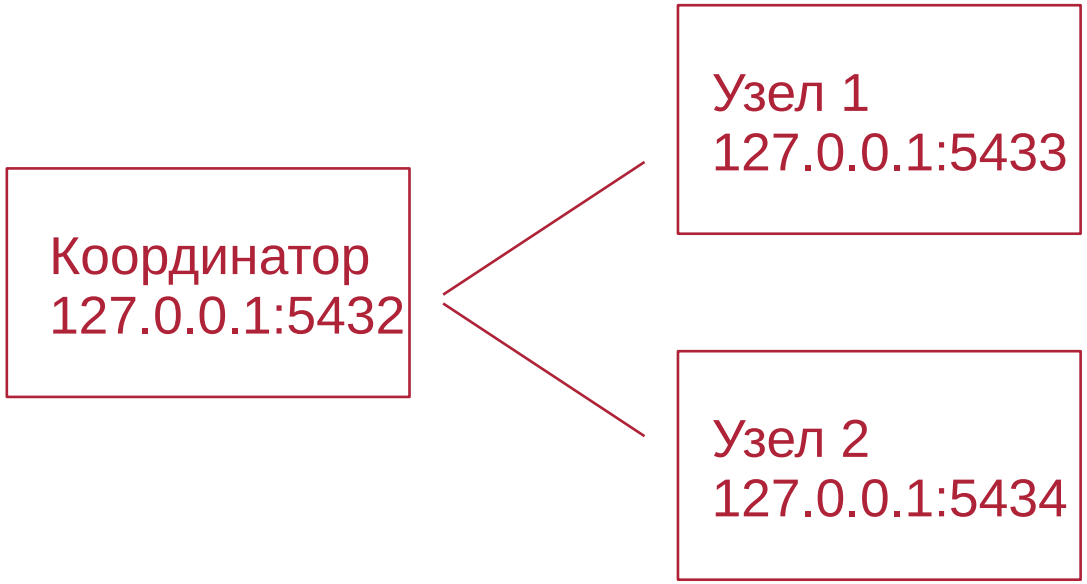




SELECT **citus_set_coordinator_host**(‘127.0.0.1’, 5432)

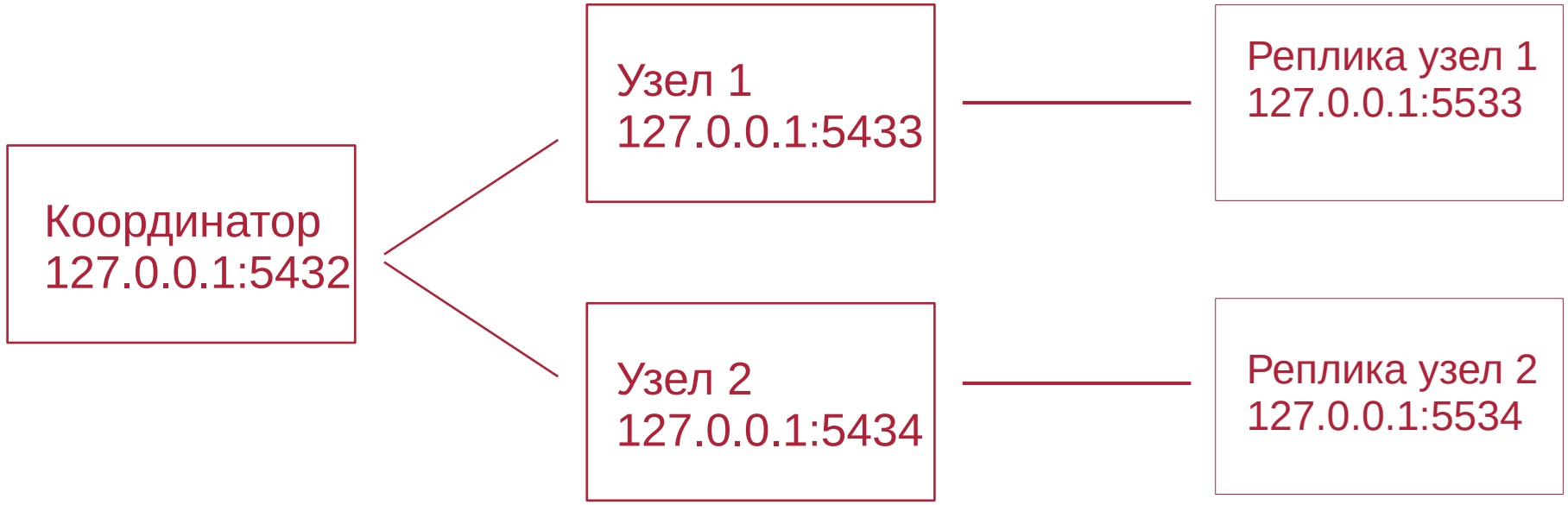
SELECT **citus_add_node** (‘127.0.0.1’, 5433)

SELECT **citus_add_node** (‘127.0.0.1’, 5434)



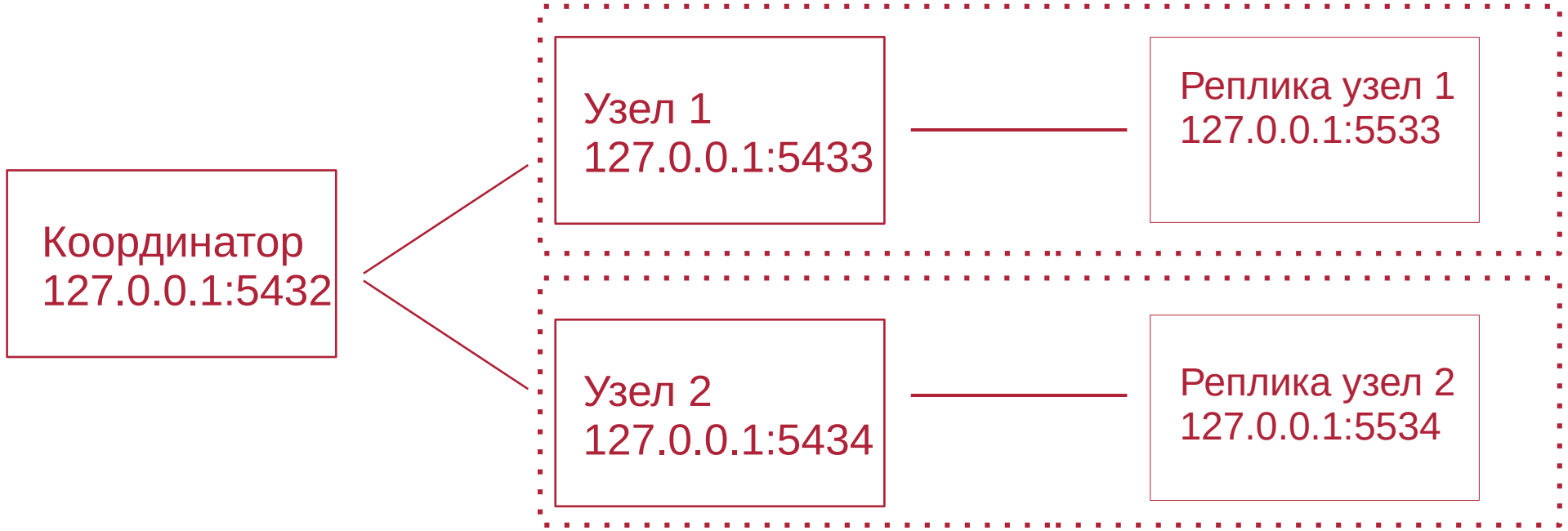
pg_dist_node

```
test=# SELECT  nodeid,groupid,nodename,nodeport,noderole
FROM pg_dist_node ;
 nodeid | groupid | nodename  | nodeport | noderole
--------+---------+-----------+----------+----------
      1 |       0 | 127.0.0.1 |     5432 | primary
      2 |       1 | 127.0.0.1 |     5433 | primary
      3 |       2 | 127.0.0.1 |     5434 | primary
(3 rows)
```



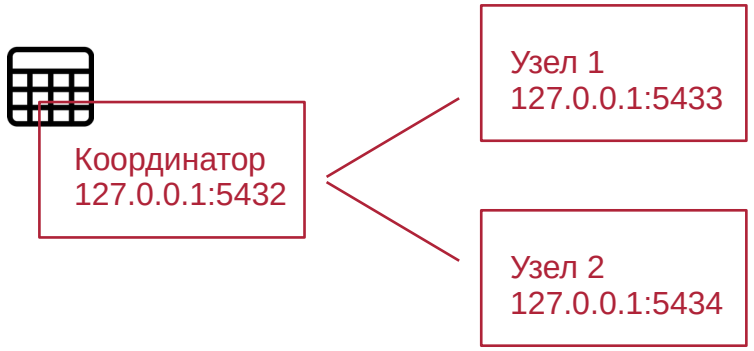
SELECT **citus_add_secondary_node** (‘127.0.0.1’, 5533,‘127.0.0.1’, 5433)

SELECT **citus_add_secondary_node** (‘127.0.0.1’, 5534,‘127.0.0.1’, 5434)



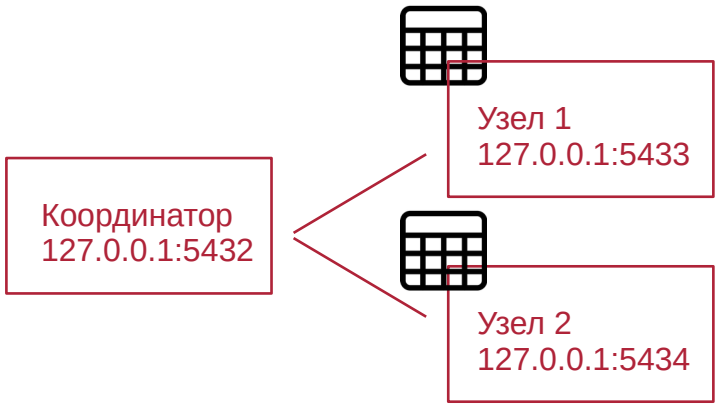

```
test=# SELECT nodeid,groupid,nodename,nodeport,noderole FROM pg_dist_node ORDER BY groupid;
 nodeid | groupid | nodename  | nodeport | noderole
--------+---------+-----------+----------+-----------
      1 |       0 | 127.0.0.1 |     5432 | primary
      2 |       1 | 127.0.0.1 |     5433 | primary
      4 |       1 | 127.0.0.1 |     5533 | secondary
      3 |       2 | 127.0.0.1 |     5434 | primary
      5 |       2 | 127.0.0.1 |     5534 | secondary
(5 rows)
```

Создаём распределённую таблицу



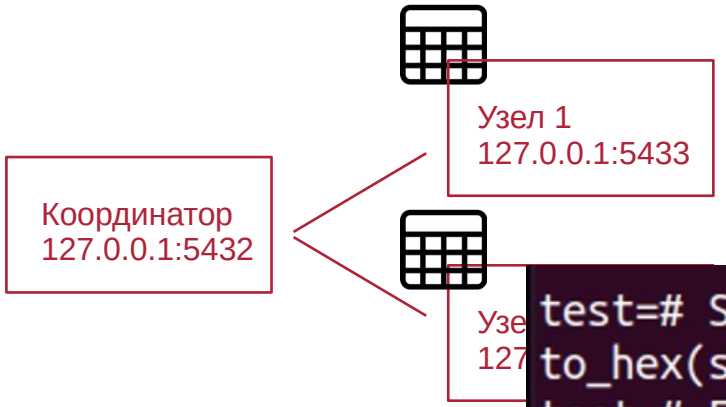
- **Нужно выбрать критерий, по которому будет происходить разделение данных**
- **Выбрать кол-во шардов, помня, что при делении шард таблица будет заблокирована**
- **Помнить, что для распределенных запросов открывается количество соединений, равное количеству шард, даже если на одном узле находится несколько шард**

Создаём распределённую таблицу



SELECT **create_distributed_table** (‘test’, ‘id’, shard_count=>2)

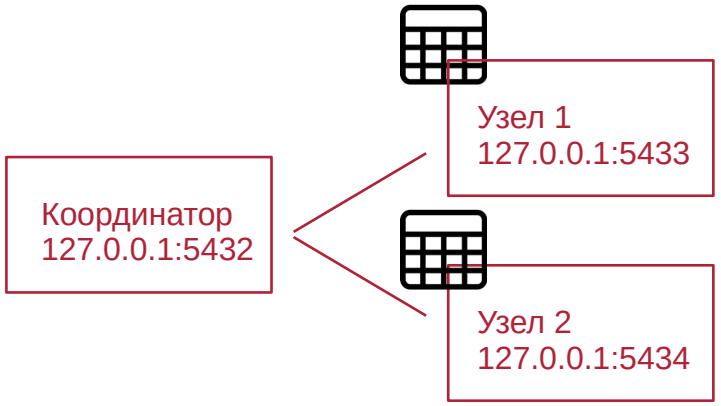
Создаём распределённую таблицу



pg_dist_shard

```
test=# SELECT logicalrelid, shardid, to_hex(shardminvalue::int) as min,
to_hex(shardmaxvalue::int)  max
test-# FROM pg_dist_shard;
 logicalrelid | shardid |   min    |   max
--------------+---------+----------+----------
 test         |  102008 | 80000000 | ffffffff
 test         |  102009 | 0        | 7fffffff
(2 rows)
```

Создаём распределённую таблицу



pg_dist_placement

```
test=# SELECT placementid,shardid,groupid
test-# FROM  pg_dist_placement;
 placementid | shardid | groupid
-------------+---------+---------
           1 |  102008 |       1
           2 |  102009 |       2
(2 rows)
```

Особенности использования serial

```
postgres@ak:/usr/local/pgsql$ psql -p 5432
psql (17.7)
Type "help" for help.

postgres=# INSERT INTO test_serial(tenant) VALUES(1);
INSERT 0 1
postgres=#
\q
postgres@ak:/usr/local/pgsql$ psql -p 5433
psql (17.7)
Type "help" for help.

postgres=# INSERT INTO test_serial(tenant) VALUES(1);
ERROR:  nextval(sequence) calls in worker nodes are not supported for column defaults of type int or smallint
postgres=#
```

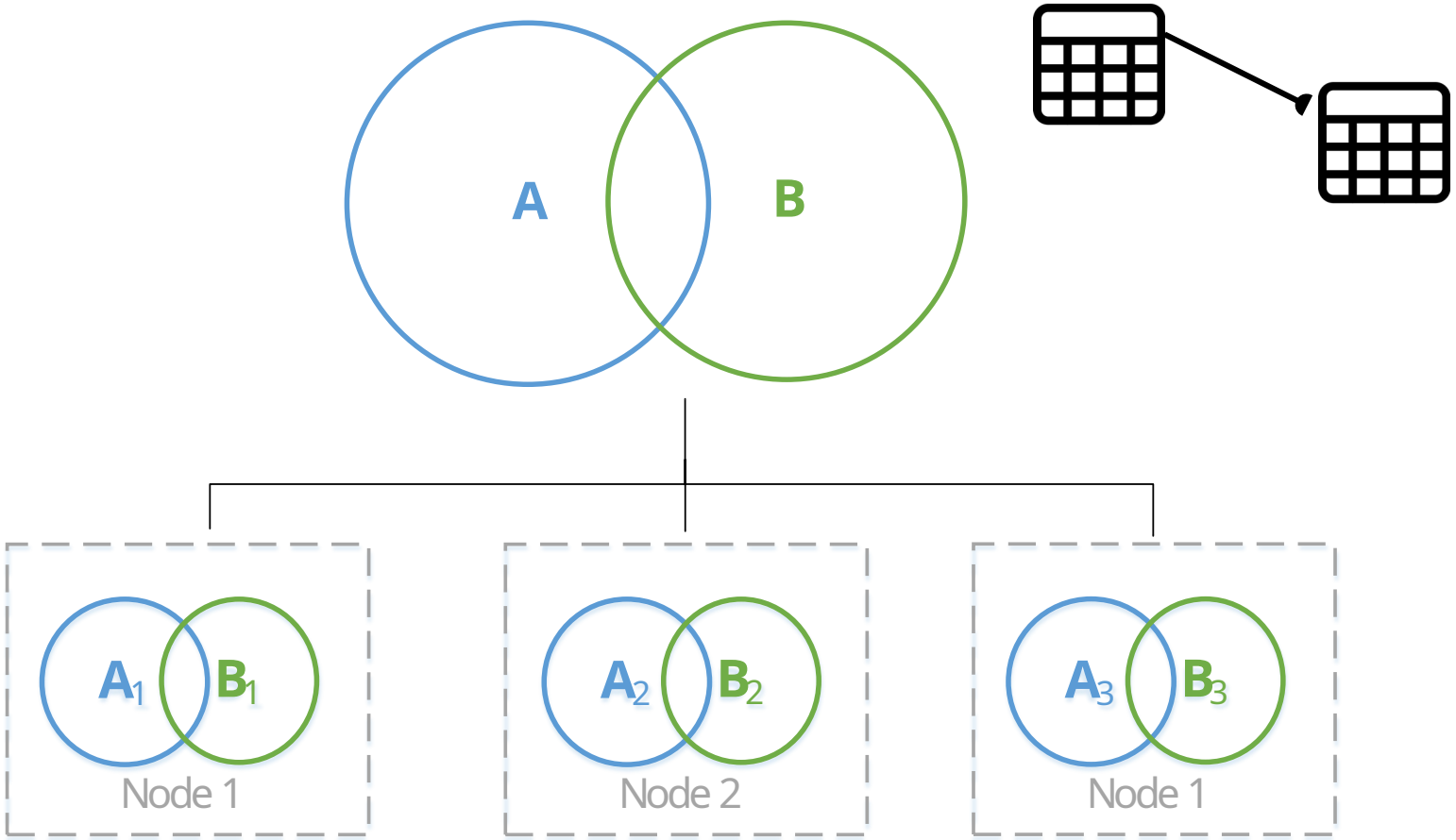
Особенности использования serial

```
postgres=# CREATE TABLE test_serial(id serial, tenant int);
CREATE TABLE
postgres=# SELECT create_distributed_table('test_serial', 'tenant', shard_count=>2);
 create_distributed_table
--------------------------

(1 row)

postgres=# \d test_serial
                              Table "public.test_serial"
 Column |  Type   | Collation | Nullable |                 Default
--------+---------+-----------+----------+-----------------------------------------
 id     | integer |           | not null | nextval('test_serial_id_seq'::regclass)
 tenant | integer |           |          |

postgres=#
\q
postgres@ak:/usr/local/pgsql$ psql -p 5433
psql (17.7)
Type "help" for help.

postgres=# \d test_serial
                                Table "public.test_serial"
 Column |  Type   | Collation | Nullable |                  Default
--------+---------+-----------+----------+-----------------------------------------------
 id     | integer |           | not null | worker_nextval('test_serial_id_seq'::regclass)
 tenant | integer |           |          |
```

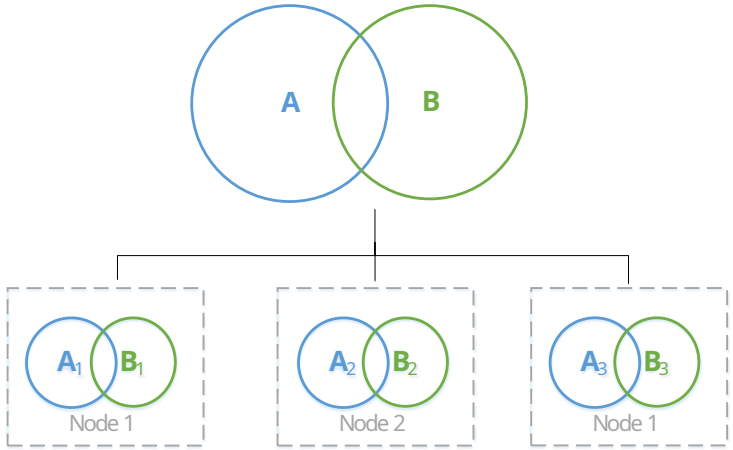
Особенности распределённых таблиц

- Колонка распределения желательно, чтобы была Primary Key
- Если установлен Primary Key – по другому столбцу данные уже не распределить
- Ограничение на тип serial, можно спользовать только через координатор
- Запрещены внешние ключи, нет union и оконных функций
- Подзапросы возможны только с соразмещенными таблицами
- Соразмещенные таблицы — вид распределенной таблицы, у которой размещение секций зависит от родительской таблицы

Соразмещённые таблицы



Соразмещённые таблицы



Если таблицы могут быть соединены по колонке распределения, то они соразмещенны

```
test=# SELECT create_distributed_table('ticket_flights','ticket_no',
test(#                                 colocate_with=>'tickets');
NOTICE:  Copying data from local table...
NOTICE:  copying the data has completed
DETAIL:  The local data in the table is no longer visible, but is still on disk
.
HINT:  To remove the local data, run: SELECT truncate_local_data_after_distribu
ting_table($$public.ticket_flights$$)
 create_distributed_table
--------------------------

(1 row)
```

Соразмещённые таблицы

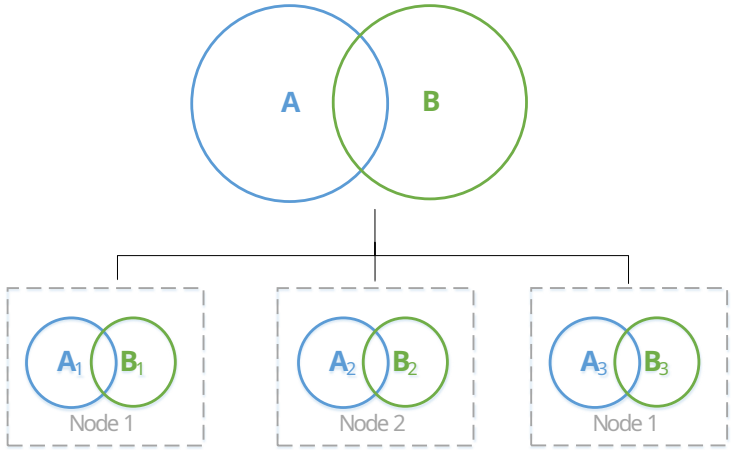
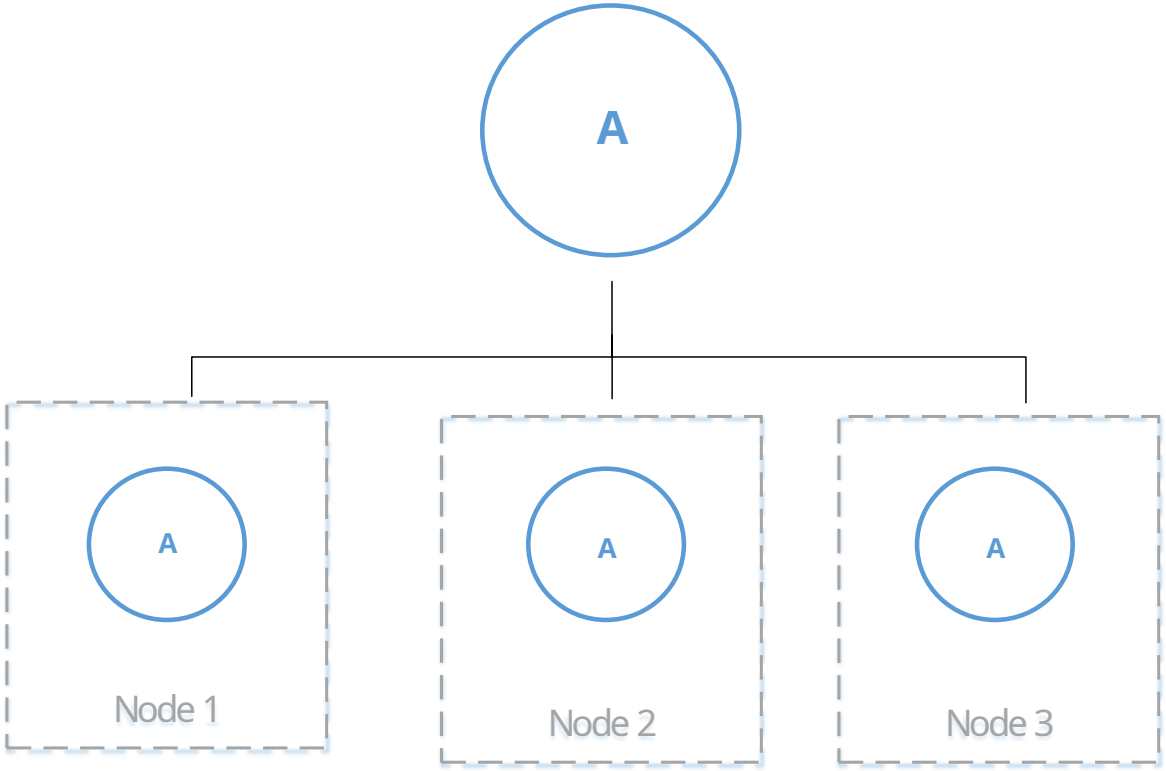


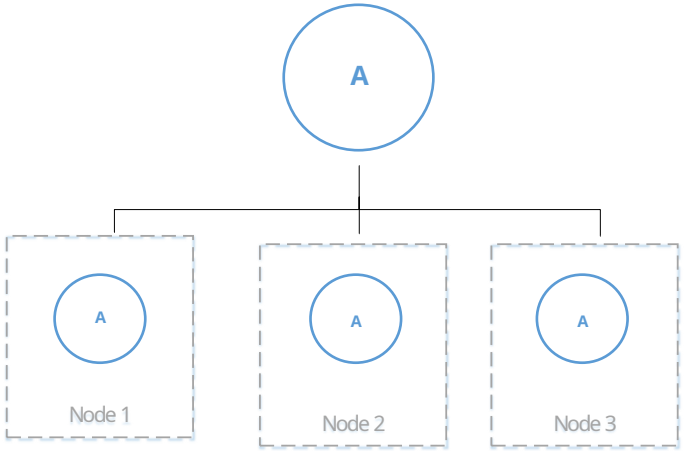
Таблица может соединяться со всеми таблицами кластера, имеющим тот же colocation_id

```
test=# SELECT table_name, citus_table_type,distribution_column,colocation_id,shard_count
test-# FROM citus_tables;
   table_name    | citus_table_type | distribution_column | colocation_id | shard_count
-----------------+------------------+---------------------+---------------+-------------
 ticket_flights  | distributed      | ticket_no           |             1 |           2
 tickets         | distributed      | ticket_no           |             1 |           2
(2 rows)
```

Справочные таблицы



Справочные таблицы



- **Таблица копируется на все рабочие узлы**
- **Может соединяться с любыми таблицами кластера**
- **Можно накладывать любые индексы**
- **Может быть изменена на любом узле, все изменения распространяются на все узлы**

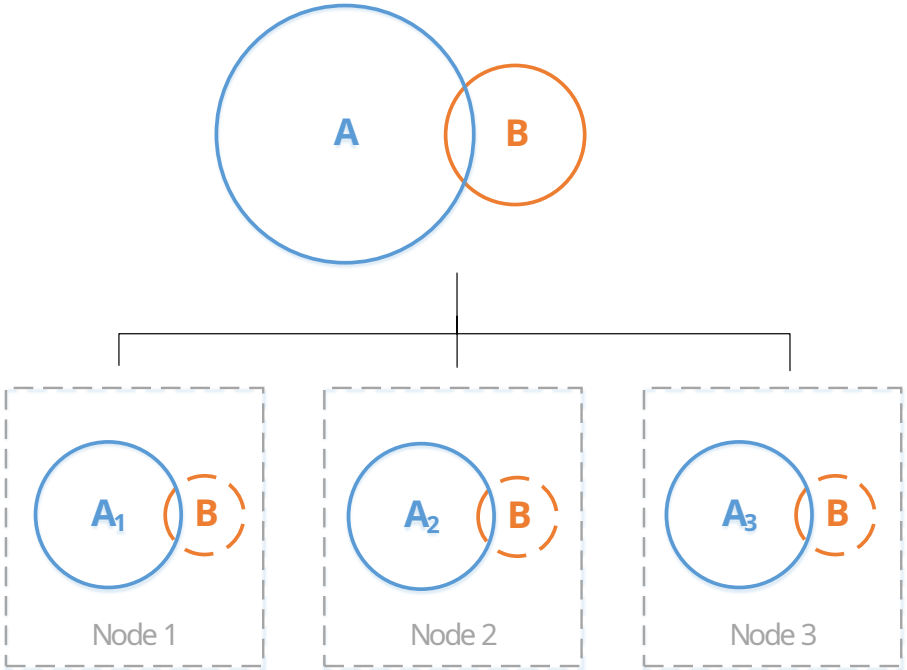
SELECT **create_reference_table**(‘flights’)

Справочные таблицы

```
test=# SELECT table_name, citus_table_type, distribution_column
test-# FROM citus_tables;
   table_name    | citus_table_type | distribution_column
-----------------+------------------+---------------------
 flights         | reference        |
 ticket_flights  | distributed      |
 tickets         | distributed      |
(3 rows)
```

```
test=# select * from pg_dist_shard;
  logicalrelid  | shardid | shardstorage | shardminvalue | shardmaxvalue
----------------+---------+--------------+---------------+---------------
 tickets        |  102008 | t            | -2147483648   | -1
 tickets        |  102009 | t            | 0             | 2147483647
 ticket_flights |  102015 | t            | -2147483648   | -1
 ticket_flights |  102016 | t            | 0             | 2147483647
 flights        |  102017 | t            |               |
(5 rows)

test=# SELECT * FROM pg_dist_placement
test-# WHERE shardid=102017 ;
 placementid | shardid | shardstate | shardlength | groupid
-------------+---------+------------+-------------+---------
          12 |  102017 |          1 |           0 |       0
          13 |  102017 |          1 |           0 |       1
          14 |  102017 |          1 |           0 |       2
(3 rows)
```

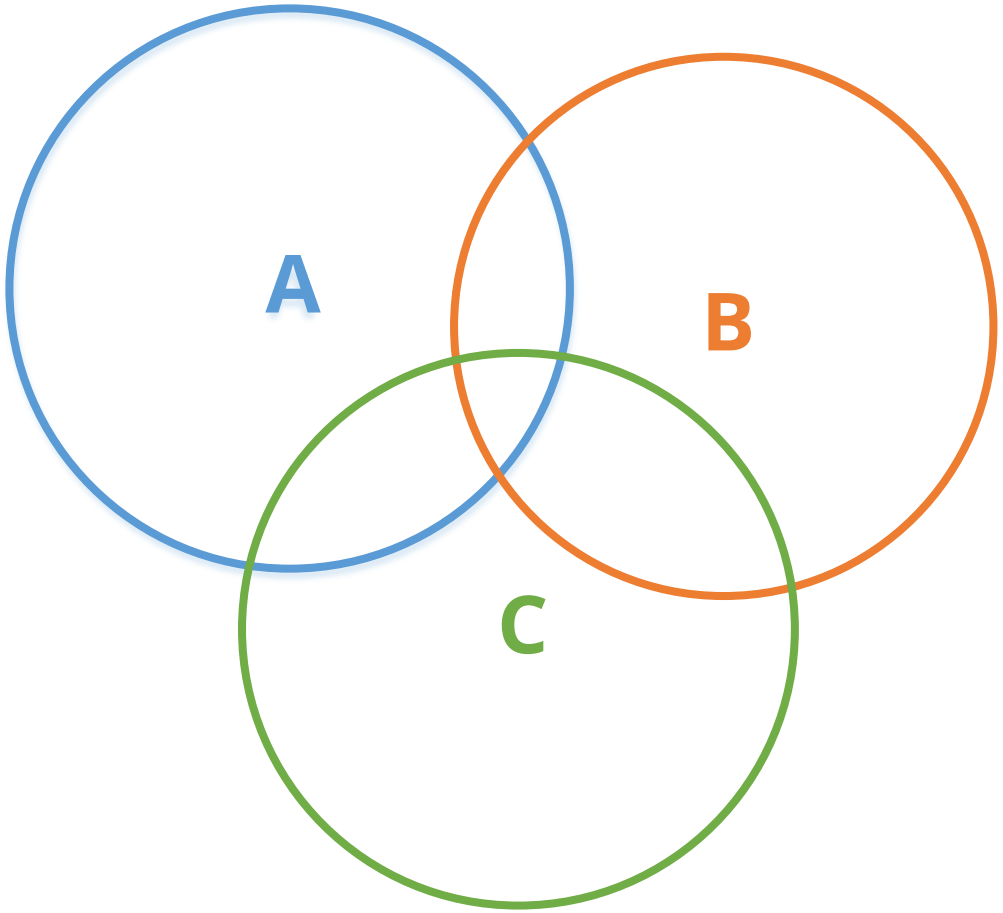
Локальные таблицы



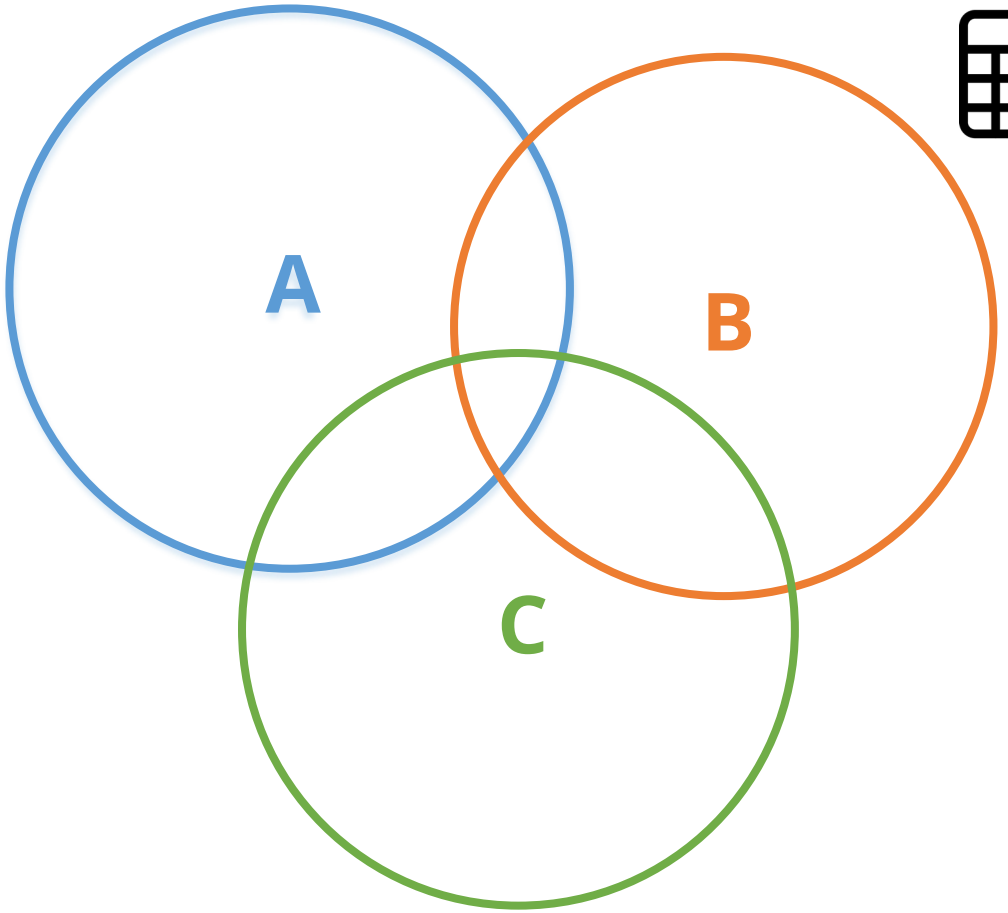

```
postgres=# SELECT * FROM pg_dist_shard;
 logicalrelid | shardid | shardstorage | shardminvalue | shardmaxvalue
--------------+---------+--------------+---------------+---------------
 test         |  102008 | t            | -2147483648   | -1
 test         |  102009 | t            | 0             | 2147483647
 test_l       |  102010 | t            |               |
(3 rows)

postgres=# SELECT * FROM pg_dist_placement;
 placementid | shardid | shardstate | shardlength | groupid
-------------+---------+------------+-------------+---------
           1 |  102008 |          1 |           0 |       1
           2 |  102009 |          1 |           0 |       2
           3 |  102010 |          1 |           0 |       0
(3 rows)
```

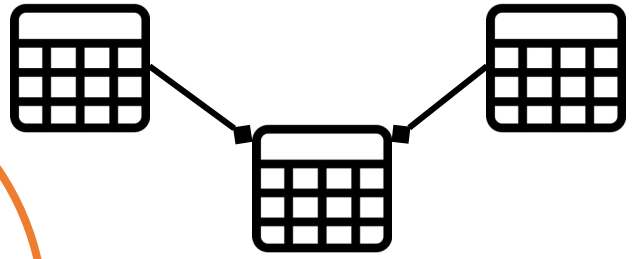
```
SELECT citus_add_local_table_to_metadata(`test_l`);
```

Отношение «многие ко многим»

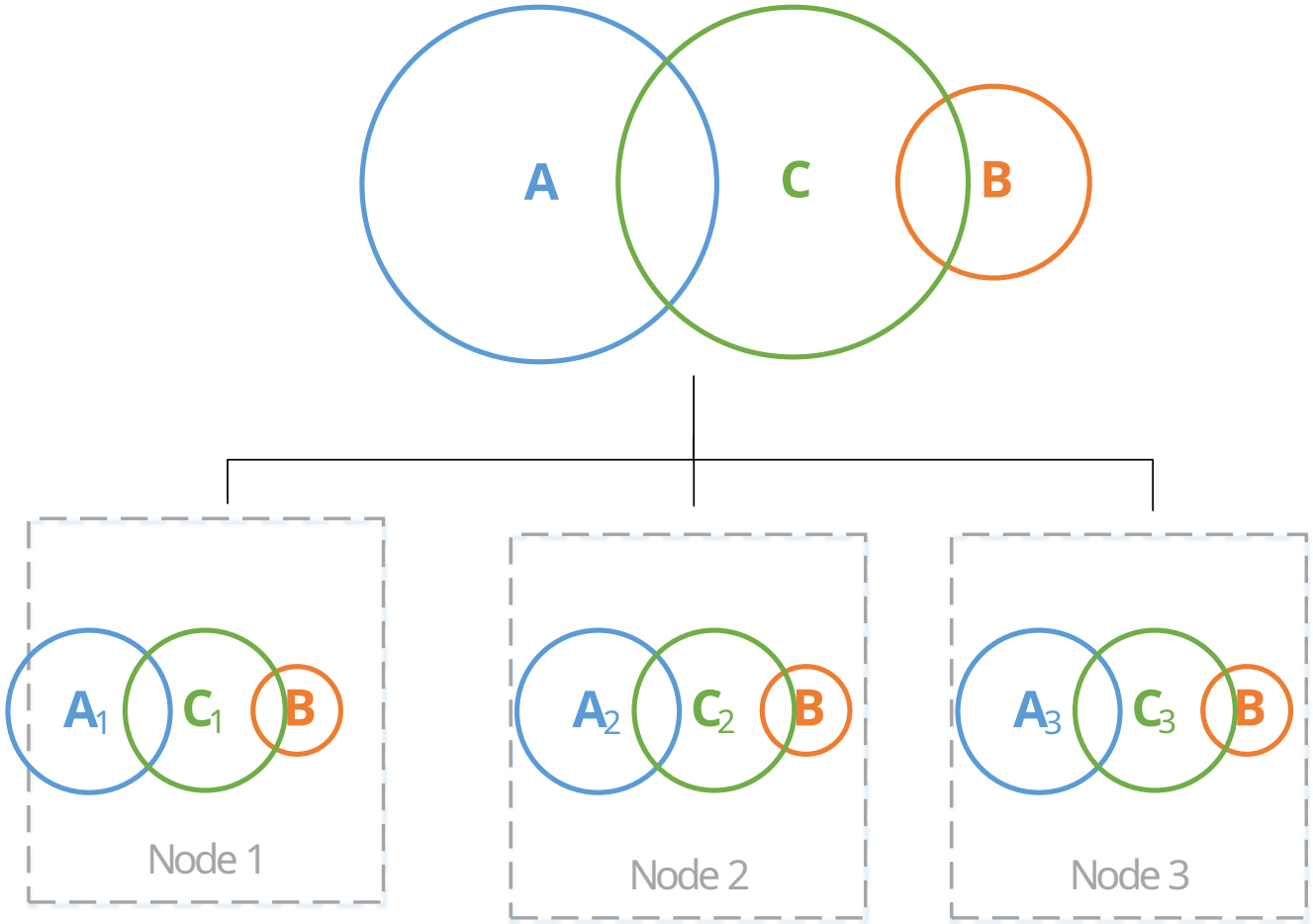


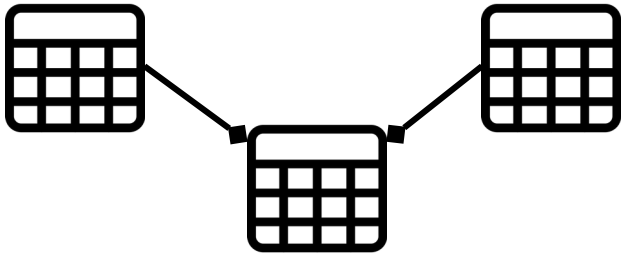
Задача распределения трех таблиц





Задача распределения трех таблиц





A
C — соразмезщенные

B — справочная

Какие еще есть метаданные

pg_dist_object
pg_dist_colocation
pg_dist_partition

```
postgres=# select * from pg_dist_colocation;
 colocationid | shardcount | replicationfactor | distributioncolumntype | distributioncolumncollation
--------------+------------+-------------------+------------------------+-----------------------------
            1 |          2 |                 1 |                     23 |                           0
(1 row)

postgres=# SELECT * from pg_dist_partition;
 logicalrelid | partmethod |
--------------+------------+-----------------------------------------
 t            | h          | {VAR :varno 1 :varattno 1 :vartype 23 :var
 t2           | h          | {VAR :varno 1 :varattno 2 :vartype 23 :var
(2 rows)
```

```
postgres=# SELECT * from pg_dist_object;
 classid | objid | objsubid | type | objec
---------+-------+----------+------+------
    1260 |    10 |        0 |      |
    1262 |     5 |        0 |      |
    2615 |  2200 |        0 |      |
    1259 | 24576 |        0 |      |
    1259 | 24584 |        0 |      |
(5 rows)
```

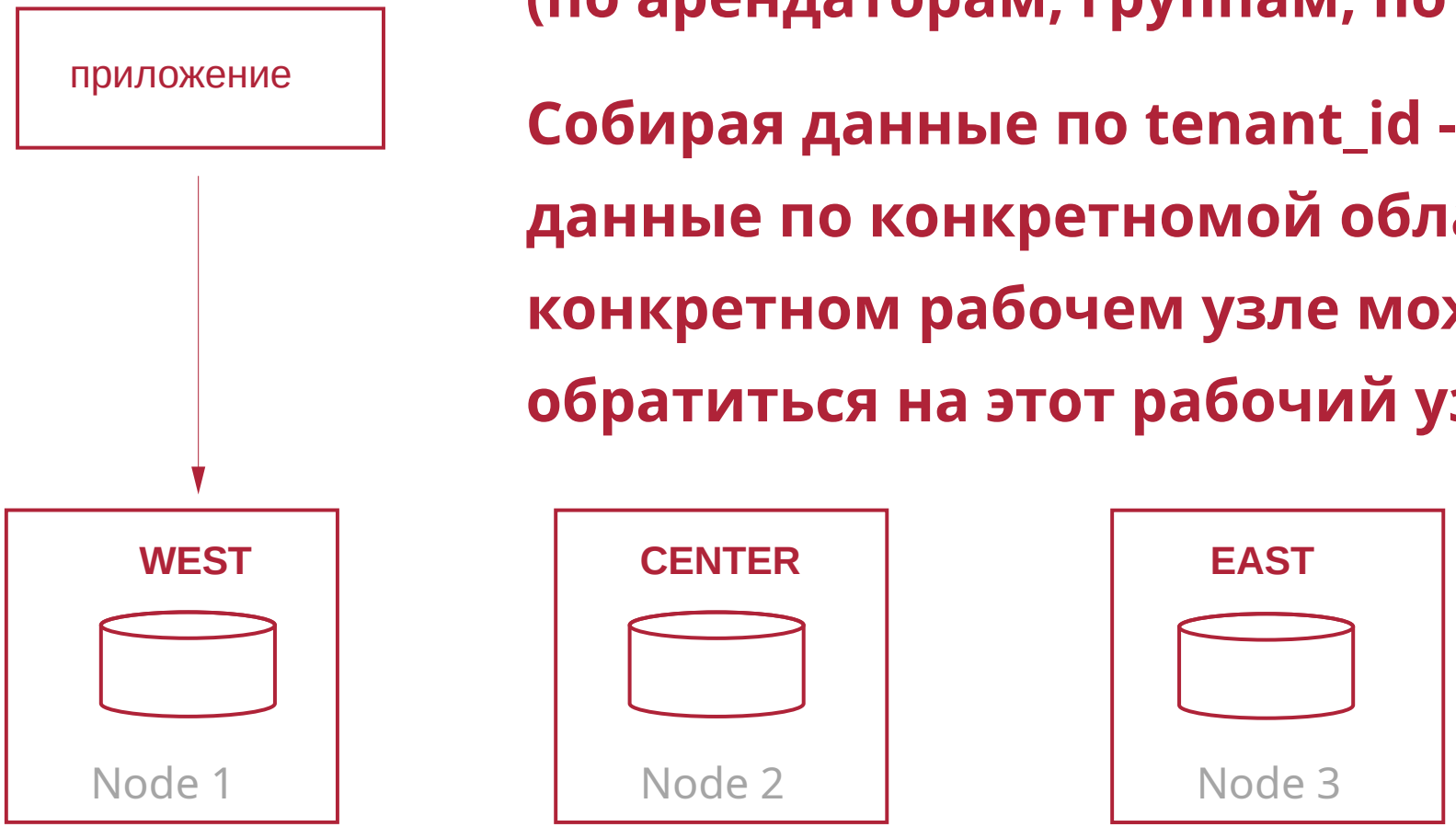
Варианты разделения данных

- По первичному ключу (рекомендуется)
- По tenant_id (по арендаторам, по группам, по областям)

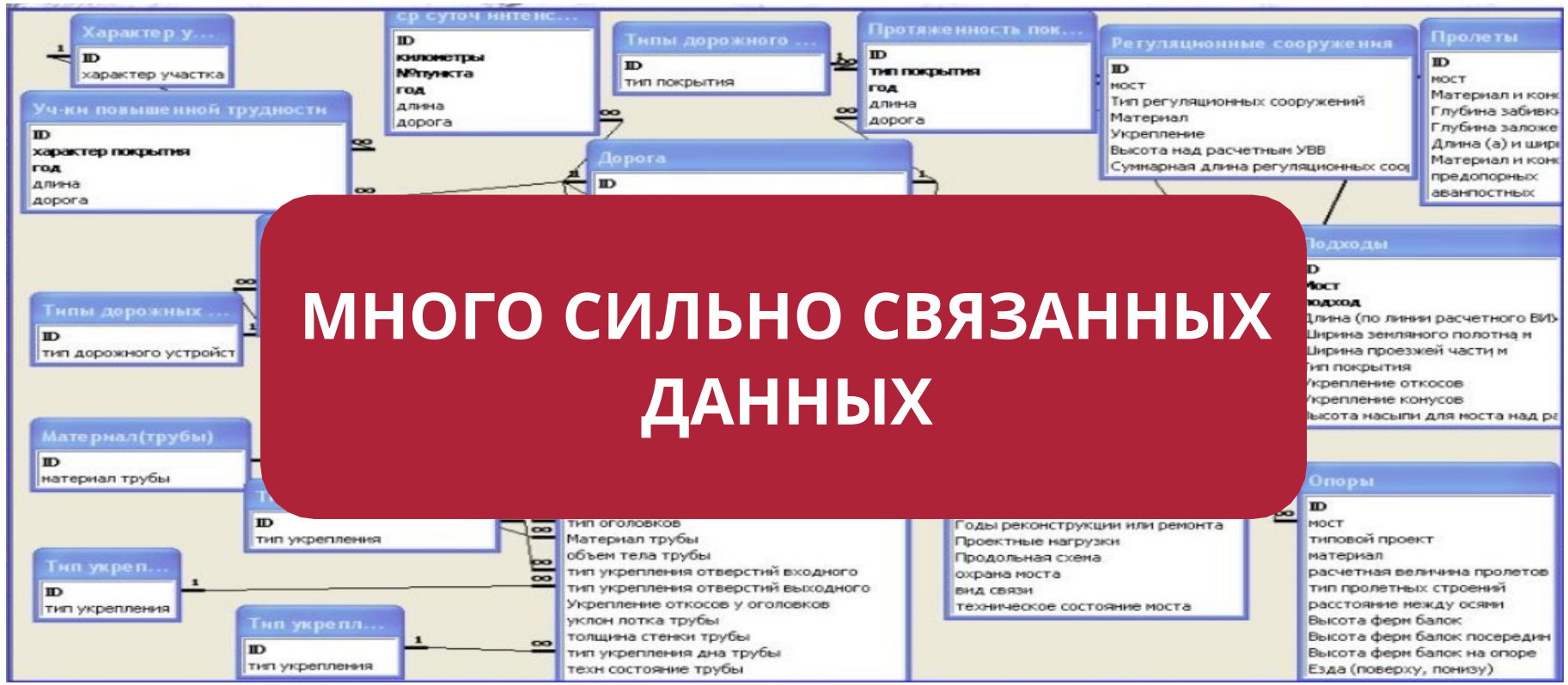
TENANT-ные приложения

Колонка разделения tenant
(по арендаторам, группам, по областям)

Собирая данные по tenant_id - мы зная, что данные по конкретномой области лежат на конкретном рабочем узле можем напрямую обратиться на этот рабочий узел.



Где не стоит использовать citus?



МНОГО СИЛЬНО СВЯЗАННЫХ ДАННЫХ

ГАЗИНФОРМ СЕРВИС

планирование и исполнение

Москва, 19 марта 2026

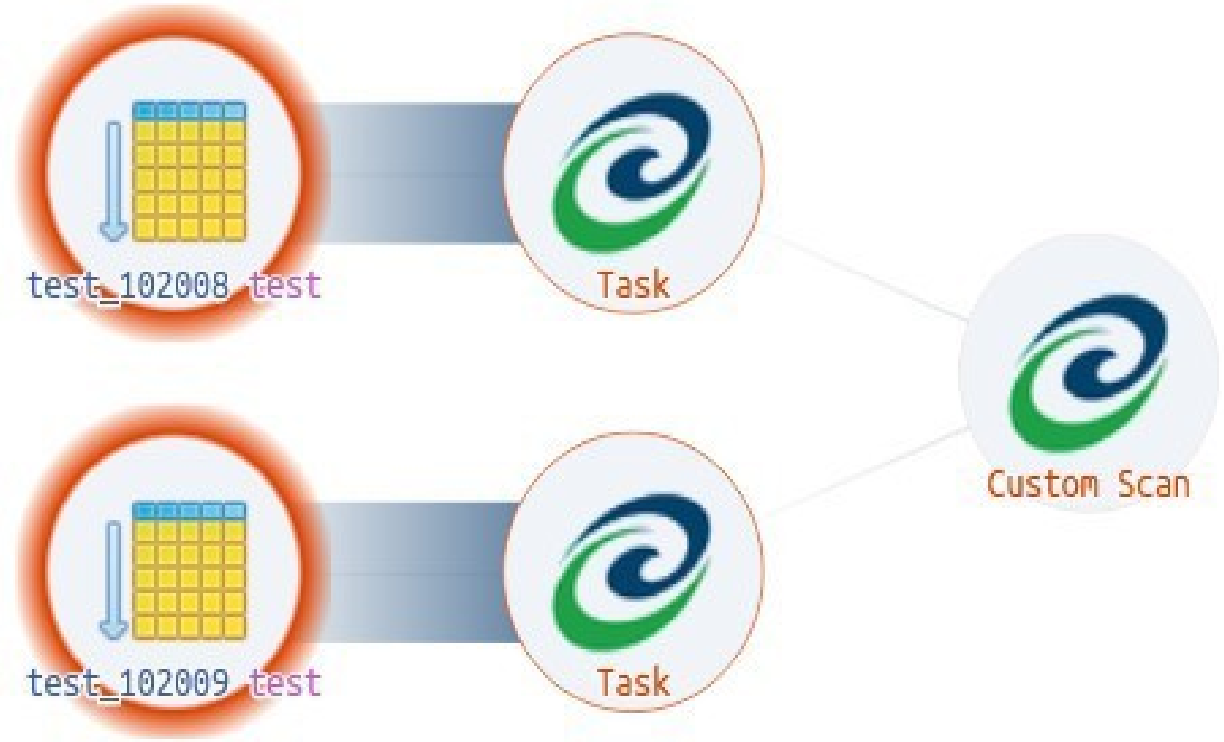
Что представляет распределенный план

SELECT * FROM test

```
test=# EXPLAIN SELECT * FROM test;
                                    QUERY PLAN
-----------------------------------------------------------------------------
 Custom Scan (Citus Adaptive)  (cost=0.00..0.00 rows=100000 width=4)
   Task Count: 2
   Tasks Shown: All
   ->  Task
         Node: host=127.0.0.1 port=5433 dbname=test
         ->  Seq Scan on test_102008 test  (cost=0.00..35.50 rows=2550 width=4)
   ->  Task
         Node: host=127.0.0.1 port=5434 dbname=test
         ->  Seq Scan on test_102009 test  (cost=0.00..35.50 rows=2550 width=4)
(9 rows)
```

Что представляет распределенный план

SELECT * FROM test



https://explain.tensor.ru

Что представляет распределенный план

SELECT * FROM test

https://explain.tensor.ru

Что представляет распределенный план

SELECT * FROM test

```
postgres@ubuntu:/usr/local/pgsql$ psql test
psql (16.6, server 16.8)
Type "help" for help.

test=# SET citus.log_remote_commands TO on;
SET
test=# SELECT * FROM test;
NOTICE:  issuing SELECT id FROM public.test_102009 test WHERE true
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 2
NOTICE:  issuing SELECT id FROM public.test_102008 test WHERE true
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 1
 id
----
  1
  2
(2 rows)
```

Планировщик

- Анализирует имя таблицы в запросе:
- Если имя таблицы с постфиксом номера шарда – строит план и отправляет на local_executor
- Иначе строит распределенный план
- Для данных, находящихся на узле, вызывается local_executor, иначе – adaptive_executor

Планировщик

```
postgres@ubuntu:/usr/local/pgsql$ psql -p 5433 test
psql (16.6, server 16.8)
Type "help" for help.

test=# SELECT id FROM public.test_102008 test;
 id
----
  1
(1 row)

test=# \q
postgres@ubuntu:/usr/local/pgsql$ psql -p 5434 test
psql (16.6, server 16.8)
Type "help" for help.

test=# SELECT id FROM public.test_102009 test;
 id
----
  2
(1 row)
```

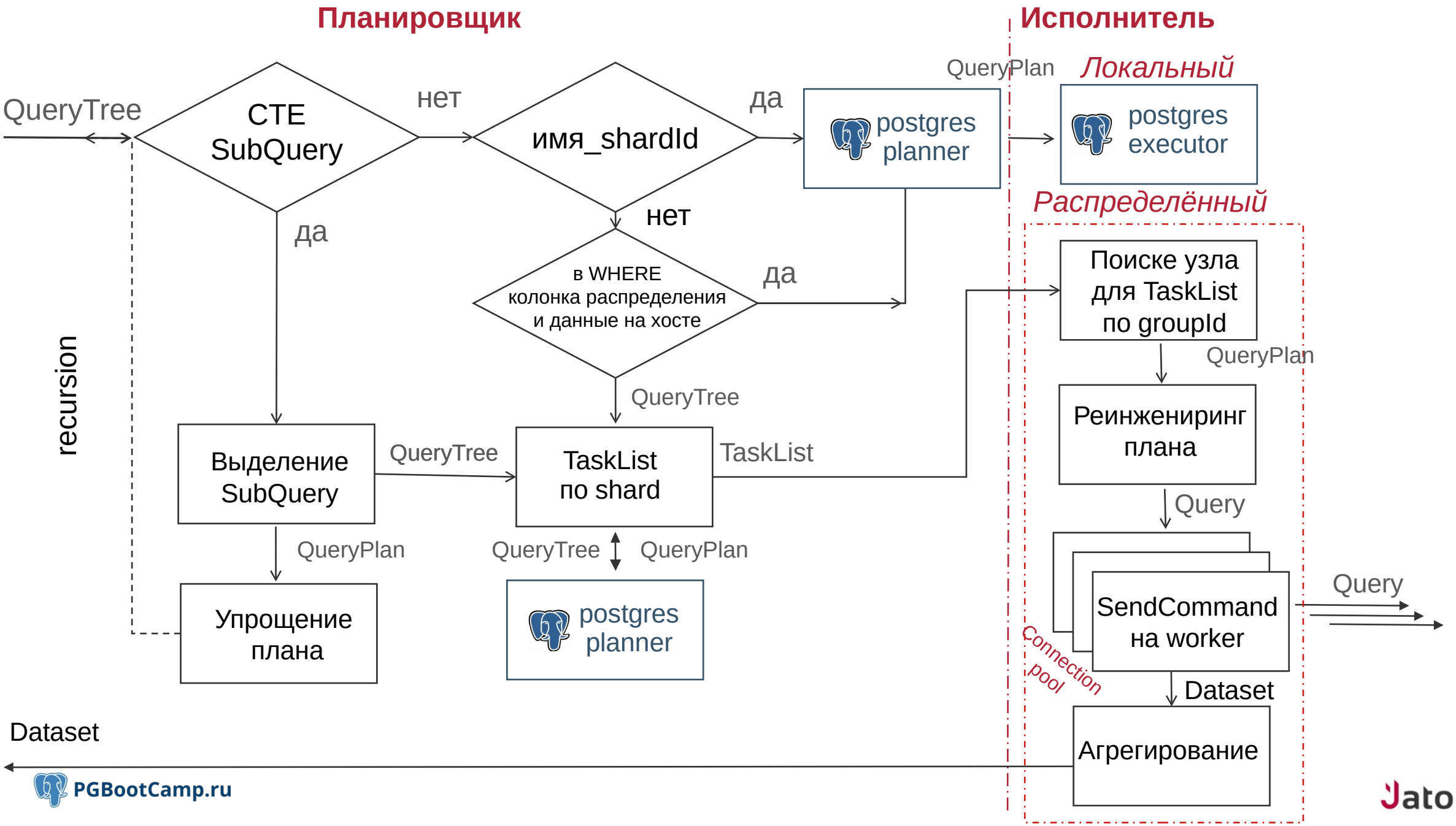
Исполнитель

Локальный исполнитель:

- исполняет запросы на узле, на который пришел запрос
- исполняет простой запрос с WHERE по колонке распределения
- исполняет DDL-операторы (ALTER, CREATE, DROP)
- выполняет модификацию справочных таблиц

Распределенный исполнитель:

- выполняет список задач через пул соединений для каждого рабочего узла
- осуществляет поиск доступного узла


Планировщик
Исполнитель
QueryTree
CTE
SubQuery
нет
имя_shardId
да
QueryPlan
Локальный
postgres
planner
postgres
executor
Распределённый
да
нет
в WHERE
колонка распределения
и данные на хосте
да
Поиске узла
для TaskList
по groupId
QueryPlan
recursion
QueryTree
Реинжениринг
плана
Выделение
SubQuery
QueryTree
TaskList
по shard
TaskList
Query
QueryPlan
QueryTree
QueryPlan
Query
SendCommand
на worker
Упрощение
плана
postgres
planner
Connection
pool
Dataset
Dataset
Агрегирование

Сложный запрос

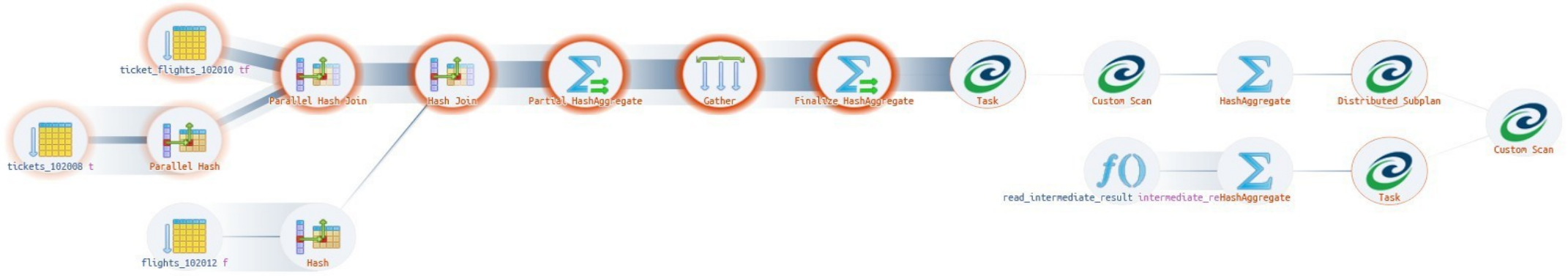

```
WITH flights_avg as (
SELECT f.flight_no,count(*), EXTRACT (day FROM scheduled_departure)
  FROM tickets t, ticket_flights tf,flights f
  WHERE t.ticket_no=tf.ticket_no AND tf.flight_id=f.flight_id GROUP BY f.flight_no,
  EXTRACT (day FROM scheduled_departure) )
SELECT flight_no,avg(count) FROM flights_avg GROUP BY flight_no;
```

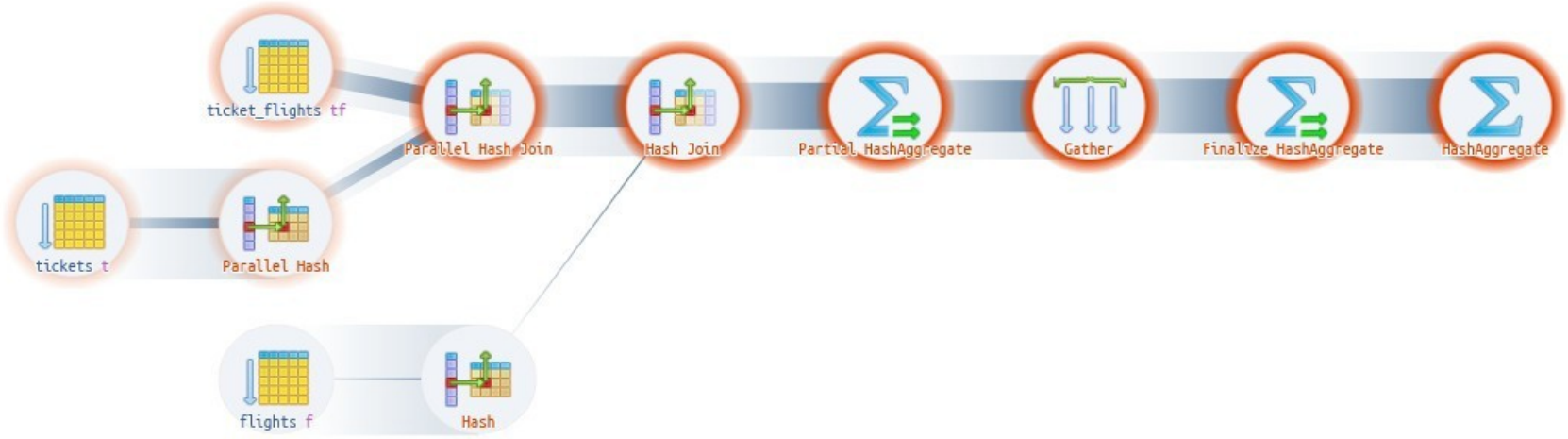
Найти среднее количество пассажиров в месяц по каждому рейсу

Подзапрос находит кол-во пассажиров за день по каждому рейсу

Внешний CTE-запрос производит агрегацию кол-ва запросов за месяц по каждому рейсу

Сравнение планов

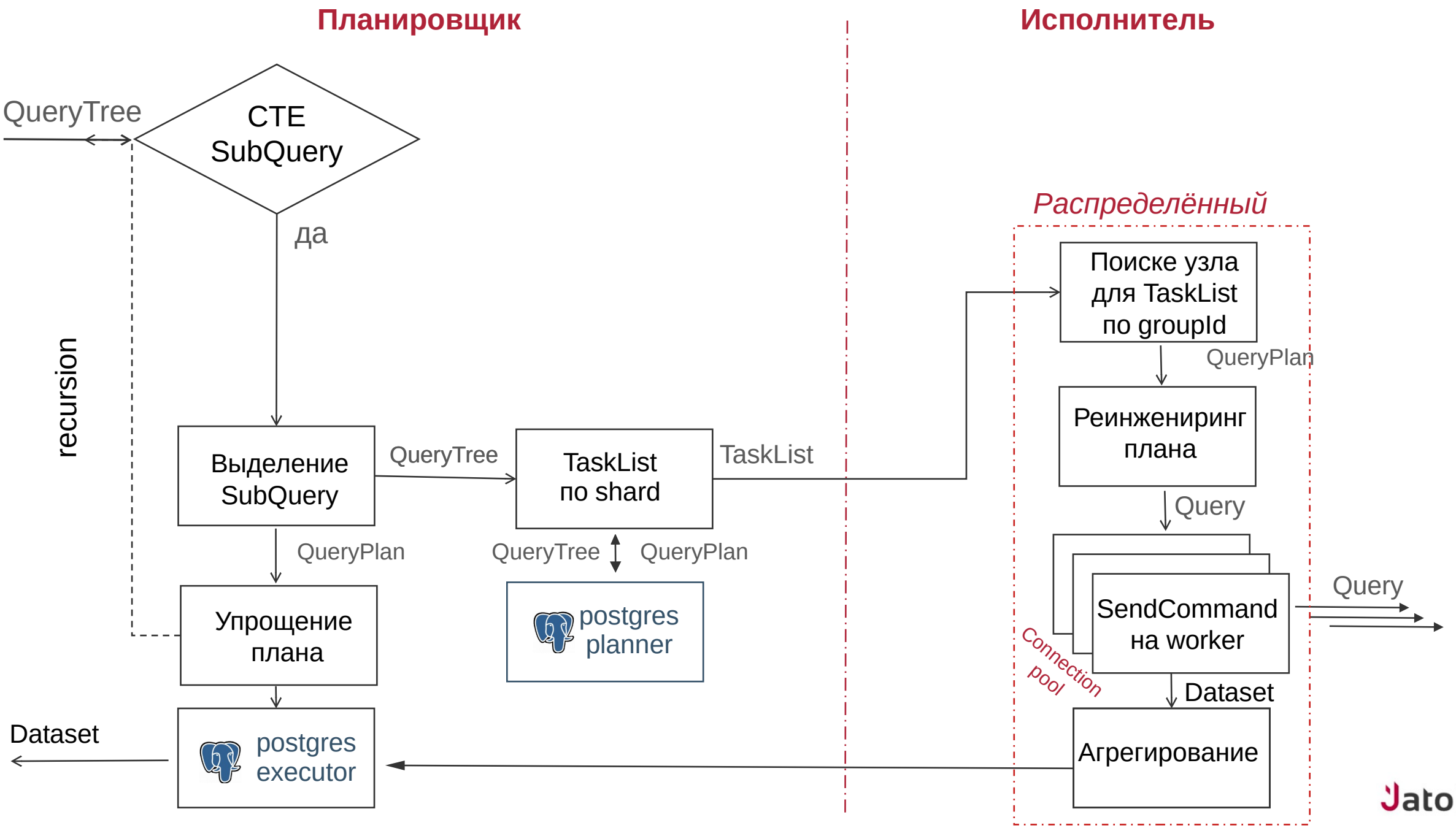




https://explain.tensor.ru

Сложный запрос


```
test=# WITH  flights_avg as  (
    SELECT f.flight_no,count(*), EXTRACT (day FROM scheduled_departure)
    FROM tickets t, ticket_flights tf,flights f
    WHERE t.ticket_no=tf.ticket_no AND tf.flight_id=f.flight_id
    GROUP BY f.flight_no, EXTRACT (day FROM scheduled_departure) )
test-# SELECT flight_no,avg(count) FROM flights_avg GROUP BY flight_no;
NOTICE:  issuing BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;SELECT assign_distributed_transaction_id(0, 3, '2025-09-08 09:13:42.434384
+03');
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;SELECT assign_distributed_transaction_id(0, 3, '2025-09-08 09:13:42.434384
+03');
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 4
NOTICE:  issuing SELECT f.flight_no, count(*) AS count, EXTRACT(day FROM f.scheduled_departure) AS "extract" FROM ((public.tickets_102019 t
JOIN public.ticket_flights_102016 tf ON ((t.ticket_no OPERATOR(pg_catalog.=) tf.ticket_no))) JOIN public.flights_102017 f ON ((tf.flight_id
OPERATOR(pg_catalog.=) f.flight_id))) WHERE true GROUP BY f.flight_no, (EXTRACT(day FROM f.scheduled_departure))
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 4
NOTICE:  issuing SELECT f.flight_no, count(*) AS count, EXTRACT(day FROM f.scheduled_departure) AS "extract" FROM ((public.tickets_102018 t
JOIN public.ticket_flights_102015 tf ON ((t.ticket_no OPERATOR(pg_catalog.=) tf.ticket_no))) JOIN public.flights_102017 f ON ((tf.flight_id
OPERATOR(pg_catalog.=) f.flight_id))) WHERE true GROUP BY f.flight_no, (EXTRACT(day FROM f.scheduled_departure))
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  executing the command locally: SELECT flight_no, avg(count) AS avg FROM (SELECT intermediate_result.flight_no, intermediate_result.
count, intermediate_result."extract" FROM read_intermediate_result('10_1'::text, 'binary'::citus_copy_format) intermediate_result(flight_no
character(6), count bigint, "extract" numeric)) flights_avg GROUP BY flight_no
NOTICE:  issuing COMMIT
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing COMMIT
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 4
 flight_no |          avg
-----------+------------------------
 PG0201    |   113.8709677419354839
```


Планировщик
Исполнитель
QueryTree
CTE
SubQuery
да
recursion
Выделение
SubQuery
QueryTree
TaskList
по shard
TaskList
QueryPlan
QueryTree
QueryPlan
Упрощение
плана
postgres
planner
Dataset
postgres
executor
Распределённый
Поиске узла
для TaskList
по groupId
QueryPlan
Реинжениринг
плана
Query
SendCommand
на worker
Query
Connection
pool
Dataset
Агрегирование

ГАЗИНФОРМ СЕРВИС

согласованность

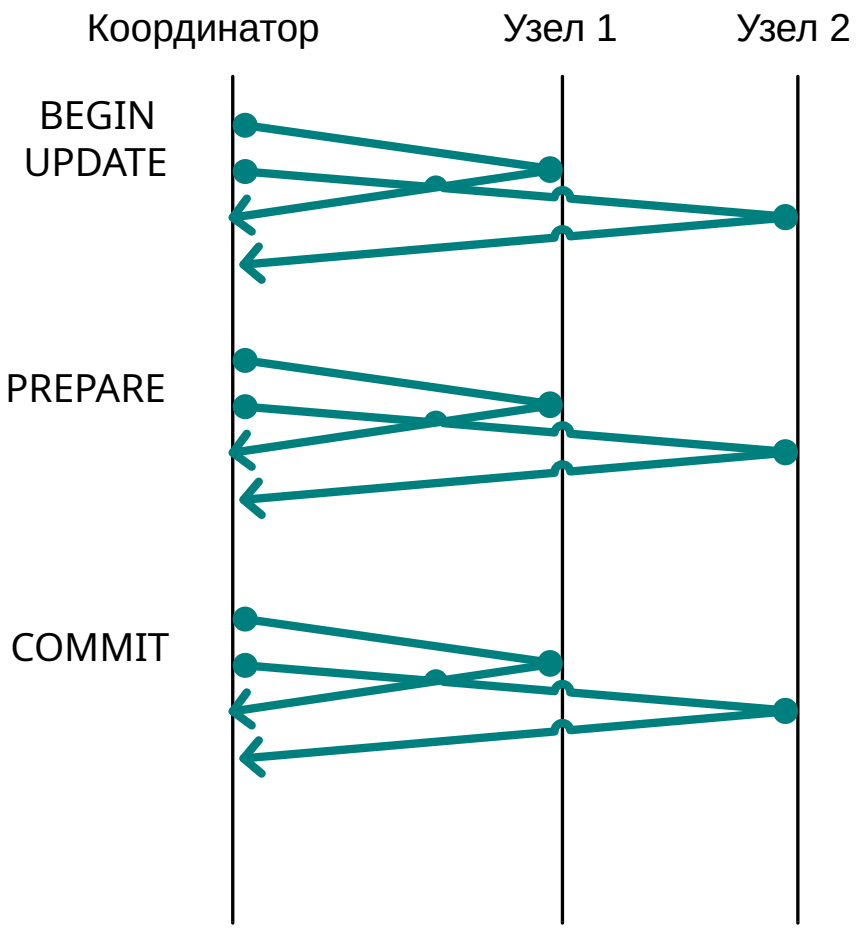
Согласованность

- Сильная согласованность в рамках одного узла
- Конечная согласованность для распределенных транзакций

- Наследует уровни изоляции PostgreSQL
- Нет global snapshot isolation между узлами, что приводит к грязному чтению

Двухфазный коммит

```
test=# UPDATE test2 SET val=2 WHERE val=1;
NOTICE:  issuing BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;SEL
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;SEL
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 7
NOTICE:  issuing UPDATE public.test2_102011 test2 SET val = 2 WHERE (
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 7
NOTICE:  issuing UPDATE public.test2_102010 test2 SET val = 2 WHERE (
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing PREPARE TRANSACTION 'citus_0_324685_3_2'
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing PREPARE TRANSACTION 'citus_0_324685_3_3'
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 7
NOTICE:  issuing COMMIT PREPARED 'citus_0_324685_3_2'
DETAIL:  on server postgres@127.0.0.1:5433 connectionId: 3
NOTICE:  issuing COMMIT PREPARED 'citus_0_324685_3_3'
DETAIL:  on server postgres@127.0.0.1:5434 connectionId: 7
UPDATE 0
test=#
```

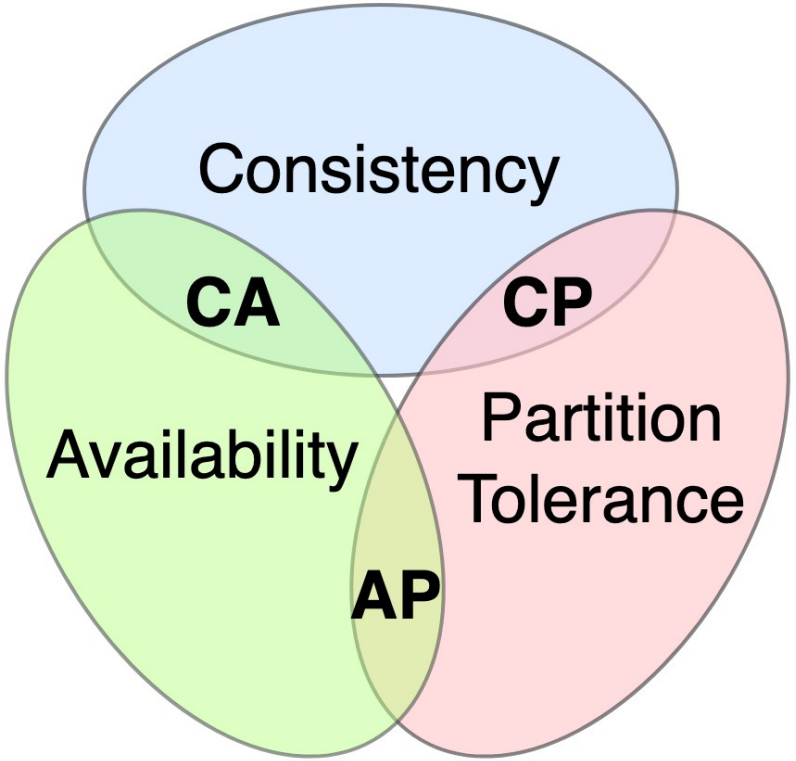


Следствие теоремы CAP

При сильной согласованности:

чем больше нагрузка

тем больше процент отказов



https://google.com

Решардинг

- Осуществляется в фоне за счет логической репликации: citus_rebalance_start()
- Таблица должна быть распределена по первичному ключу или настоить REPLICA IDENTY
- Есть возможность использовать свою стратегию ребаланса
- Можно переместить конкретный шард: citus_move_shard_placement()

Деление таблицы на шарды

Можно изменить кол-во шард: **alter_distributed_table()**

Осуществляется через блокирующую операцию копирования

ГАЗИНФОРМ СЕРВИС

отказоустойчивость

Отказоустойчивость

pg_autofailover

https://github.com/hapostgres/pg_auto_failover

Patroni

https://github.com/patroni/patroni

CitusFailover

https://github.com/akalend/citus

jaDog

http://jatoba.ru

Citus - Failover

Основные идеи:

- **Должно работать из коробки, вместе с citus**
- **Простота в настройке**
- **Каждый узел мониторит свою группу**

Citus - Failover

- ICMP (ping)	~0.05 ms
- UDP пакеты 64b	~0.5 ms
- TCP (Pqping)	~4.5 ms

Citus - Failover

```
postgres=# select * from citus_health();
WARNING:  172.17.0.5:5432 nodeid=5
 pid | ready |      send       | errors | packages |  ttl
-----+-------+-----------------+--------+----------+-------
  51 | t     | 172.17.0.5:5432 |      0 |      364 | 0.354
(1 row)
postgres=#
postgres=# select pid,application_name ,backend_type  from pg_stat
_activity WHERE backend_type ~ 'Citus';
 pid |     application_name     |          backend_type
-----+--------------------------+-------------------------------
  51 | UDP sender               | Citus Health Sender:5432
  52 | UDP server               | Citus Health UDP:5432
  54 | Citus Maintenance Daemon | Citus Maintenance Daemon: 5/10
(3 rows)
```

Что почитать

- **Исходный код** https://github.com/citusdata/citus
- **Документация Microsoft** http://clck.ru/3S8A6V
- **Сравнение Citus и ArenaData** https://clck.ru/3S8Jas
- **Создание масштабируемой и высокодоступной системы Postgres с помощью Patroni 3.0 и Citus (Хабр)** https://clck.ru/3S8BD2
- **Шардирование. Паттерны и антипаттерны. HL++14** http://clck.ru/3S89wd
- **А.Петров. Распределенные данные.** СПб. Питер, 2022.

ГАЗИНФОРМ СЕРВИС

вопросы?

Александр Календарёв
разработчик СУБД Jatoba